

Ревью кода на JS

Основные цели code review

- Выявление ошибок и их устранение на стадии, где это можно сделать безболезненно.
- Улучшение качества кода. Здесь речь идет о стандартах, принятых внутри компании, отказе от дублирования и мертвого кода.
- Обмен опытом – заглядываем в будущее. Перенять знания и не возвращаться к своим ошибкам в дальнейшем – главное преимущество коллективной работы и тут на помощь нам снова приходит review.
- Совместное владение кодом - исключает ситуацию, что код может быть переписан или удален с фразой - "Ничего не понятно, надо все переписать", если его автор, например, покинул компанию.
- Качественный продукт является конечной целью вашей работы. А review дает преимущество в её достижении. Именно так его и нужно воспринимать. Полученные замечания в результате review – всего лишь повод не допустить их попадания в релиз.

Как подготовиться к review

Давайте представим, что вам нужно продать свой проект. В дальнейшем кто-то другой будет заниматься его обслуживанием. В таком случае все должно быть максимально прозрачно. Помимо этого, хорошим решением будет все проверить. Держите эту идею в голове перед review и следуйте простым рекомендациям.

Соблюдайте стандарт и пишите документацию к модулям

Пишите по [стандарту](#), соблюдайте правила [документирования](#) (достаточно краткой документации, хотя бы описание, что за модуль и зачем нужен).

Соблюдайте основные правила

Используйте в коде только:

1. Logicless шаблонизатор.
2. Новые контролы.
3. Новые рекордсеты и источники данных.

Не используйте в коде deprecated зависимости.

Следите за количеством строк в методе

Допускается писать методы не более 200 строк, но в идеале метод не должен превышать 50 строк. Если не уместается – разбивайте на несколько мелких методов, решающих подзадачи основного метода.

Удалите неиспользуемый код

Избавьтесь от частей кода с пометкой "fixme", "удалить", "костыль", "не используется" или добавьте ссылки на задачи, по которым этот код будет удален. Если в проекте есть код, который не используется, проект не будет принят.

Избавьтесь от дублирования

Здесь все просто – чем меньше дублей, тем более структурированным будет ваш код. Тем самым вы уменьшите его объем, и упростите поддержку в будущем.

Не допускайте утечки памяти

К сожалению, утечки на глаз не видно (чаще всего), поэтому здесь мы говорим о подписках на глобальные объекты и не дочерние контролы – отписывайтесь.

Пример:

Другими словами, если у вас есть код:

```
this.getParent().subscribe...
```

То

добавьте:

```
this.getParent().unsubscribe...
```

Отписываться от дочерних контролов нет необходимости. Ссылка на них исчезнет вместе с подпиской и родителем.

Берегите ресурсы нашего ЦОД и клиентов

Помним, что все объявленные переменные, которые в дальнейшем не используются, будут удаляться сборщиком мусора.

Например, вы объявили `i` в целях отладки:

```
function()
{
  var i = 0; /*Далее i не используется*/
}
```

В таком случае не должно быть объявления переменной - после завершения работы функции `i` будет удалена из памяти сборщиком мусора.

Еще одна лишняя работа - описание методов «пустышек».

Пример: в пустом `init` вызывается базовый `init`. При наследовании каждый такой метод копируется впустую.

Пишите комментарии

Наличие комментариев помогает ускорить процесс восприятия чужого кода. Но, важно помнить, комментарий должен передавать логику работы метода. Не пишите в комментарии то, что можно выразить языком программирования.

Пример того, как делать не надо:

```
// Проверим, есть ли контрол на форме
if (this.hasChildControlByName('ctr1') ) {...}
```

Хороший комментарий к данному примеру:

```
//Контрола может не быть на форме, если форма строится на my.sbis.ru, а там нет компонентов документооборота
```

Следите за качеством кода

Чтобы избежать потребности «перевода» с языка программирования, используйте осмысленные имена переменных, функций и методов.

Описание разметки должно быть вынесено в отдельный TEMPL-файл. Генерирование верстки в JS-файле недопустимо.

Придерживайтесь единого стиля проекта.

Придерживайтесь модульного подхода к review

Модульное review подразумевает поэтапную сдачу проекта. Отдельные части программы отправляются на review по мере их выполнения. Если на момент готовности проекта его этапы приняты, то фактически останется только показать работу программы в целом.

Проверьте код перед отправкой на review

Перед отправкой кода на review проверьте его самостоятельно. Для этого используйте автоматический анализатор кода — `code_reviewer.py`.

Утилита проверяет соответствие клиентского кода некоторым требованиям сдачи проекта и принимает опционально следующие аргументы:

Review_project

- Только Logicless шаблонизатор.
- Только новые контролы (из старых контроллов SBIS3.CORE допустимо использовать Infobox, HTMLView, FloatArea, Dialog, Paging, Control, CompoundControl, Master).
- Только новые рекордсеты и источники данных.
- Отсутствие \$ws в коде.

- Отсутствие deprecated зависимостей.

Find_duplicate

Построение отчета по дублированию кода. Для поиска дублирования в коде по папке утилита вызывает СТАН (работает с SDK 3.17.220).

Скачать скрипт можно [по ссылке](#).

Пример вызова:

```
python3 code_reviewer.py ~/my-project/dir1 --sdk=3.17.220 --review_project --find_duplicate
```

Если версию SDK не указать, то поиск дублирования в коде вызван не будет.

Правильно организуйте инфраструктуру интерфейсного модуля.

В [статье](#) приведены правила, предъявляемые к организации любых интерфейсных модулей. Данный набор правил позволит вам корректно спроектировать интерфейсный модуль, благодаря чему он будет готов к платформенным алгоритмам паковки и раздельного обновления. Нововведения связаны с [проектом отказа от оглавления](#).

Требования к интерфейсным модулям актуальны только для проектов, запущенных в стадию «Выполнить» начиная с декабря 2017 года.